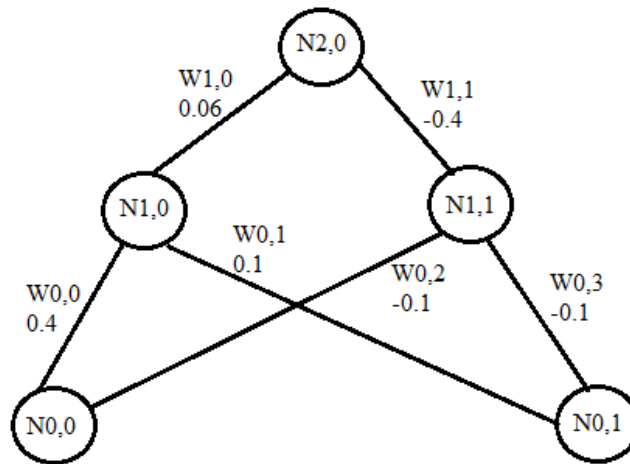


Propagación hacia adelante (forward propagation) o reetroalimentación (feed forward)

La propagación hacia adelante es el proceso que propaga la señal desde la capa de entrada a la capa de salida o visual. Considera esta red de tres capas:



La salida deseada de N2,0 es 1.0. Las entradas son [1, 1] (N0,0 y N0,1). La tasa de aprendizaje es $\beta = 0.45$ y el impulso o momentum es $\alpha = 0.9$.

```
> beta <- 0.45
> alpha <- 0.9
> input <- N0 <- matrix(c(1,1))
> w0 <- matrix(c(.4,-.1,.1,-.1), nrow=2)
> print(input)
      [,1]
[1,]    1
[2,]    1
> print(w0)
      [,1] [,2]
[1,]  0.4  0.1
[2,] -0.1 -0.1
```

Cada una de las neuronas ocultas y de salida son **neuronas logísticas**, lo que significa que aplican la función logística

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

```
> sigma <- function(t) 1/(1+exp(-t))
```

a la entrada antes de devolver la respuesta.
Calcula las entradas en la capa oculta.

```
> N1 <- sigma(w0 %*% input)
> print(N1)
      [,1]
[1,] 0.6224593
[2,] 0.4501660
```

Calcula las entradas en la capa de salida.

```
> w1 <- matrix(c(0.06, -0.4), nrow=1)
> print(w1)
      [,1] [,2]
[1,] 0.06 -0.4
> N2 <- sigma(w1 %*% N1)
```

La salida es:

```
> N2
      [,1]
[1,] 0.4643807
```

Retropropagación

Primero, calculamos el error en N2,0.

Sustituir:

```
> N2.0.error <- N2 * (1-N2) * (1-N2)
> print(N2.0.error)
      [,1]
[1,] 0.1332253
```

Una vez que se conoce el error, se utilizará para la propagación hacia atrás y el ajuste de pesos.

Calcula la tasa de cambio de los pesos en cada uno de los dos nodos con la ecuación:

```
> w1.Rate = (beta * N2.0.error[1,1]) * t(N1)
> print(w1.Rate)
      [,1] [,2]
[1,] 0.03731729 0.02698807
```

```
> print(w1)
      [,1] [,2]
[1,] 0.06 -0.4
```

Calcula los nuevos pesos. t es la iteración de la propagación hacia atrás. Obviamente, la primera vez, no se aplica la tasa de aprendizaje α , pero en iteraciones posteriores, los múltiplos de la tasa de aprendizaje se aplican en cada iteración.

```
> t <- 1
> w1.new <- w1 + w1.Rate + alpha*(t-1)
> print(w1.new)
      [,1] [,2]
[1,] 0.09731729 -0.3730119

> N1.0.error <- N2.0.error %*% w1.new
```

Calcula la tasa de cambio de los pesos entre la entrada y la capa oculta.

```
> w0.Rate = t(beta * N1.0.error) * (N0)
> print(w0.Rate)
      [,1]
[1,] 0.005834304
[2,] -0.022362575
```

Nota: $w0.Rate[1]$ se aplica a los pesos $W0.[1,2]$ y $w0.Rate[2]$ se aplica a $W0.[2,3]$

```
> w0.Rate <- matrix(c(w0.Rate[1,1], w0.Rate[2,1], w0.Rate[1,1], w0.Rate[2,1]), nrow=2)
> print(w0.Rate)
      [,1] [,2]
[1,] 0.005834304 0.005834304
[2,] -0.022362575 -0.022362575
```

Calcula los nuevos pesos en la capa de entrada.

```
> w0.new <- w0 + w0.Rate + alpha*(t-1)
> print(w0.new)
      [,1] [,2]
[1,] 0.4058343 0.1058343
[2,] -0.1223626 -0.1223626
```

Segunda iteración

Ejecuta la propagación hacia adelante nuevamente para ver si mejoramos.

```
> w0 <- w0.new
> N1 <- sigma(w0 %*% input)
> w1 <- w1.new
> N2 <- sigma(w1 %*% N1)
> print(N2)
      [,1]
[1,] 0.4742839
>
> N2.1.error <- N2 * (1-N2) * (1-N2)
> print(N2.1.error)
      [,1]
[1,] 0.1310814
```

El error mejora de 0.1332253 a 0.1310814. Esto no es una gran mejora, pero como se señaló anteriormente, la tasa de aprendizaje aumentará y aumentará la convergencia hacia los pesos adecuados.